

```
In [ ]: import random
import marimo as mo
from ortools.sat.python import cp_model
```

TP1.2 - Sudoku genérico como problema de restrições

O mesmo modelo resolve grelhas $n^2 \times n^2$. Cada célula é uma variável inteira entre 1 e n^2 . Linhas, colunas, blocos e pistas são apenas grupos de células; o modelo aplica a todos a mesma regra `AllDifferent` e fixa os valores já conhecidos.

Objetivo: encontrar uma atribuição que satisfaça as restrições. Não há função objetivo a otimizar nem exigência de solução única.

```
In [ ]: class box:
    """Grupo arbitrário de células; None significa que uma célula está livre."""

    def __init__(self, n, cells=None):
        if isinstance(n, bool) or not isinstance(n, int) or n < 1:
            raise ValueError("n tem de ser um inteiro positivo")
        self.n = n
        self.size = n * n
        self.cells = {}
        if cells is not None:
            for (i, j), value in cells.items():
                self.add(i, j, value)

    def add(self, i, j, val=None):
        if (isinstance(i, bool) or isinstance(j, bool)
            or not isinstance(i, int) or not isinstance(j, int)
            or not (0 <= i < self.size and 0 <= j < self.size)):
            raise ValueError("coordenadas fora da grelha")
        if val is not None and (isinstance(val, bool)
                                or not isinstance(val, int)
                                or not 1 <= val <= self.size):
            raise ValueError("valor fora do intervalo [1, n²]")
        old = self.cells.get((i, j))
        if old is not None and val is not None and old != val:
            raise ValueError("a célula já tem outro valor fixo")
        self.cells[(i, j)] = old if old is not None else val
        return self

    def matrix(self):
        result = [[0] * self.size for _ in range(self.size)]
        for (i, j), value in self.cells.items():
            if value is not None:
                result[i][j] = value
        return result
```

```
In [ ]: class cube(box):
    """Bloco n por n identificado pelos índices (i, j) dos blocos."""

    def __init__(self, n, i, j):
        super().__init__(n)
        if (isinstance(i, bool) or isinstance(j, bool)
            or not isinstance(i, int) or not isinstance(j, int)
            or not (0 <= i < n and 0 <= j < n)):
            raise ValueError("índices de bloco inválidos")
        for row in range(i * n, (i + 1) * n):
            for col in range(j * n, (j + 1) * n):
                self.add(row, col)
```

```
In [ ]: class path(box):
    """Troço horizontal ou vertical, com os dois extremos incluídos."""

    def __init__(self, n, inicio, fim):
        super().__init__(n)
        if len(inicio) != 2 or len(fim) != 2:
            raise ValueError("cada extremo precisa de linha e coluna")
        i0, j0 = inicio
        i1, j1 = fim
        if i0 != i1 and j0 != j1:
            raise ValueError("o troço tem de ser horizontal ou vertical")
        if i0 == i1:
            step = 1 if j1 >= j0 else -1
            for j in range(j0, j1 + step, step):
                self.add(i0, j)
        else:
            step = 1 if i1 >= i0 else -1
            for i in range(i0, i1 + step, step):
                self.add(i, j0)
```

```
In [ ]: def random_clues(n, k=None, rng=None):
    """Escolhe k posições distintas e valores independentes em [1, n²]."""
    clues = box(n)
    if k is None:
        k = n
    if isinstance(k, bool) or not isinstance(k, int) or not 0 <= k <=
clues.size**2:
        raise ValueError("k tem de estar entre 0 e n⁴")
    if rng is None:
        rng = random.Random()
    for position in rng.sample(range(clues.size**2), k):
        i, j = divmod(position, clues.size)
        clues.add(i, j, rng.randint(1, clues.size))
    return clues
```

```
In [ ]: class SudokuCSP:
    """Modelo genérico: recebe grupos, sem conhecer a origem de cada um."""

    def __init__(self, n):
        if isinstance(n, bool) or not isinstance(n, int) or n < 1:
            raise ValueError("n tem de ser um inteiro positivo")
```

```

self.n = n
self.size = n * n
self.model = cp_model.CpModel()
self.grid = [
    [self.model.new_int_var(1, self.size, f"x_{i}_{j}")
     for j in range(self.size)]
    for i in range(self.size)
]
self.group_count = 0
self.status = None
self.solve_seconds = None
self.branches = None

def add_groups(self, *groups):
    for group in groups:
        if not isinstance(group, box) or group.n != self.n:
            raise ValueError("todos os grupos têm de usar o mesmo n")
        variables = [self.grid[i][j] for i, j in group.cells]
        if variables:
            self.model.add_all_different(variables)
        for (i, j), value in group.cells.items():
            if value is not None:
                self.model.add(self.grid[i][j] == value)
        self.group_count += 1

def solve(self, time_limit=10.0):
    """Devolve a grelha, None se for impossível, ou erro se
    inconclusivo."""
    solver = cp_model.CpSolver()
    solver.parameters.max_time_in_seconds = time_limit
    solver.parameters.num_search_workers = 1
    status = solver.solve(self.model)
    self.status = solver.status_name(status)
    self.solve_seconds = solver.wall_time
    self.branches = solver.num_branches
    if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        return [[solver.value(x) for x in row] for row in self.grid]
    if status == cp_model.INFEASIBLE:
        return None
    raise RuntimeError(f"o solver não concluiu: {self.status}")

```

```

In [ ]: def make_sudoku(n, clues, extra_groups=()):
    """Monta as N linhas, N colunas, n² blocos e as pistas."""
    size = n * n
    groups = []
    for i in range(size):
        groups.append(path(n, (i, 0), (i, size - 1)))
        groups.append(path(n, (0, i), (size - 1, i)))
    for i in range(n):
        for j in range(n):
            groups.append(cube(n, i, j))
    groups.append(clues)
    groups.extend(extra_groups)
    problem = SudokuCSP(n)

```

```
problem.add_groups(*groups)
return problem
```

```
In [ ]: def validate_solution(n, solution, clues):
        """Validação independente do solver: dimensões, grupos e pistas."""
        size = n * n
        expected = set(range(1, size + 1))
        assert solution is not None and len(solution) == size
        assert all(len(row) == size for row in solution)
        for i in range(size):
            assert set(solution[i]) == expected, f"linha {i} inválida"
            assert {solution[j][i] for j in range(size)} == expected, f"coluna
{i} inválida"
        for bi in range(n):
            for bj in range(n):
                values = {
                    solution[bi * n + di][bj * n + dj]
                    for di in range(n) for dj in range(n)
                }
                assert values == expected, f"bloco {(bi, bj)} inválido"
        for (i, j), value in clues.cells.items():
            if value is not None:
                assert solution[i][j] == value, f"pista {(i, j)} alterada"
        return True
```

```
In [ ]: def format_grid(grid, n):
        """Representação textual com separadores de blocos."""
        size = n * n
        width = len(str(size))
        rows = []
        for i, row in enumerate(grid):
            parts = [
                " ".join(f"{value:>{width}}" for value in row[j:j + n])
                for j in range(0, size, n)
            ]
            rows.append(" | ".join(parts))
            if i % n == n - 1 and i + 1 < size:
                rows.append("-" * len(rows[-1]))
        return "\n".join(rows)
```

```
In [ ]: def run_example(n, seed, max_attempts=20):
        """Repete a amostragem se pistas independentes criarem um caso
impossível."""
        rng = random.Random(seed)
        for attempt in range(1, max_attempts + 1):
            clues = random_clues(n, rng=rng)
            problem = make_sudoku(n, clues)
            solution = problem.solve()
            if solution is not None:
                validate_solution(n, solution, clues)
                return clues, solution, problem, attempt
        raise RuntimeError("não foi possível obter pistas compatíveis nas
tentativas")
```

```
In [ ]: def example_markdown(n, result):
        clues, solution, problem, attempt = result
        return (
            f"## Exemplo: n={n}, grelha {n*n}x{n*n}\n\n"
            f"Pistas geradas em {attempt} tentativa(s):\n\n"
            f"```\text\n{format_grid(clues.matrix(), n)}\n```\n\n"
            f"Solução validada:\n\n"
            f"```\text\n{format_grid(solution, n)}\n```\n\n"
            f"Grupos: {problem.group_count}; estado: `{problem.status}`; "
            f"tempo do solver: {problem.solve_seconds:.4f} s; "
            f"ramos: {problem.branches}."
        )
```

Abstrações e modelo

`box(n)` guarda um dicionário (linha, coluna) -> valor ou `None`. `cube` preenche um bloco; `path` preenche um segmento reto inclusive, nos dois sentidos. A matriz é apenas uma forma de apresentar o grupo: zero significa célula fora dele ou ainda sem valor.

`random_clues` escolhe posições sem repetição e valores independentes. Esses valores podem tornar o CSP impossível; nesse caso `solve()` devolve `None`. Nos exemplos abaixo repetimos a amostragem para exibir uma solução. Isto **não** garante que o puzzle tenha solução única.

Escolhi CP-SAT do OR-Tools porque usa variáveis inteiras finitas e fornece diretamente `AddAllDifferent`, a restrição central do exercício. As classes de grupos não têm código do solver; o solver só recebe conjuntos de células.

```
In [ ]: example_2 = run_example(2, seed=2026)
```

Exemplo: $n=2$, grelha 4×4

Pistas geradas em 1 tentativa(s):

0 0 | 0 1

0 2 | 0 0

0 0 | 0 0

0 0 | 0 0

Solução validada:

4 3 | 2 1

1 2 | 3 4

2 1 | 4 3

3 4 | 1 2

Grupos: 13; estado: **OPTIMAL** ; tempo do solver: 0.0022 s; ramos: 192.

```
In [ ]: example_3 = run_example(3, seed=2027)
```

Exemplo: n=3, grelha 9×9

Pistas geradas em 1 tentativa(s):

```
0 0 0 | 0 0 0 | 0 0 0
0 7 0 | 0 1 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
```

```
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
```

```
0 0 0 | 0 0 6 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
0 0 0 | 0 0 0 | 0 0 0
```

Solução validada:

```
3 4 5 | 6 9 7 | 8 1 2
6 7 8 | 4 1 2 | 3 9 5
2 1 9 | 3 5 8 | 7 6 4
```

```
9 2 4 | 7 8 5 | 1 3 6
8 6 7 | 1 3 4 | 2 5 9
1 5 3 | 2 6 9 | 4 7 8
```

```
4 3 1 | 5 2 6 | 9 8 7
5 8 2 | 9 7 1 | 6 4 3
7 9 6 | 8 4 3 | 5 2 1
```

Grupos: 28; estado: **OPTIMAL** ; tempo do solver: 0.0250 s; ramos: 6492.

```
In [ ]: example_4 = run_example(4, seed=2028, max_attempts=1)
```

Teste adicional de escala: n=4, grelha 16×16 validada; 49 grupos; 0.1940 s e 45929 ramos nesta execução. Este tempo não garante o mesmo desempenho noutros puzzles.

Testes de correção e caso impossível

Além das grelhas resolvidas acima, verifico limites, caminhos nos dois sentidos e um par de pistas contraditórias na mesma linha. Um caso impossível é distinto de um limite de tempo atingido: o primeiro devolve **None** ; o segundo causa uma exceção, para não parecer uma prova de insatisfatibilidade.

```
In [ ]: def must_reject(action):
        try:
            action()
        except ValueError:
            return
        raise AssertionError("era esperado ValueError")

must_reject(lambda: box(2).add(4, 0))
must_reject(lambda: box(2).add(0, -1))
must_reject(lambda: box(2).add(0, 0, 0))
must_reject(lambda: box(2).add(0, 0, 5))
must_reject(lambda: path(2, (0, 0), (1, 1)))
assert len(path(2, (0, 3), (0, 0)).cells) == 4
assert len(path(2, (3, 0), (0, 0)).cells) == 4
assert len(cube(2, 1, 1).cells) == 4
assert len(random_clues(2, 4, random.Random(10)).cells) == 4

impossible = box(2).add(0, 0, 1).add(0, 1, 1)
impossible_model = make_sudoku(2, impossible)
assert impossible_model.solve() is None
assert impossible_model.status == "INFEASIBLE"
mo.md("**Testes adicionais:** limites, direções, grupos e UNSAT — todos passaram.")
```

Testes adicionais: limites, direções, grupos e UNSAT — todos passaram.

Extensão pequena: Sudoku diagonal

A modelação não muda. Basta juntar mais dois grupos `box`, um por diagonal, e aplicar `add_groups` como a qualquer outro grupo. Isto demonstra que a abstração aceita novas regras sem duplicar a lógica de resolução.

```
In [ ]: diagonal_main = box(2)
diagonal_other = box(2)
for i in range(4):
    diagonal_main.add(i, i)
    diagonal_other.add(i, 3 - i)
diagonal_problem = make_sudoku(2, box(2), (diagonal_main, diagonal_other))
diagonal_solution = diagonal_problem.solve()
assert validate_solution(2, diagonal_solution, box(2))
assert {diagonal_solution[i][i] for i in range(4)} == set(range(1, 5))
assert {diagonal_solution[i][3 - i] for i in range(4)} == set(range(1, 5))
mo.md(f"``text\n{format_grid(diagonal_solution, 2)}\n``")
```

```
1 2 | 4 3
3 4 | 2 1
-----
2 1 | 3 4
4 3 | 1 2
```

Estrutura, custo e limites

Escrevendo $N = n^2$, há $N^2 = n^4$ variáveis, cada uma com domínio de N valores. O Sudoku base cria $3N$ grupos `AllDifferent` (linhas, colunas e blocos), cada um com N células. Construir esses grupos e a matriz de variáveis custa $O(N^2)$ em espaço e tempo, antes da procura. `box.matrix()` custa $O(N^2)$; um `cube` ou `path` completo custa $O(N)$.

A procura de uma solução CSP não tem limite polinomial geral. Uma busca ingênua poderia explorar até N^{N^2} atribuições; a propagação e a pesquisa do CP-SAT reduzem muito o trabalho na prática, mas os tempos medidos acima pertencem apenas a estas instâncias e a esta máquina.

Os exemplos gerados têm poucas pistas e normalmente admitem várias soluções. Este trabalho resolve e valida uma delas; não afirma unicidade. Para um puzzle insatisfável, o resultado é `None`. Se o solver não concluir dentro do limite, lança-se um erro em vez de afirmar `UNSAT`.